

UNIVERSITAS NEGERI YOGYAKARTA
FAKULTAS TEKNIK
PROGRAM STUDI TEKNIK INDUSTRI

LAPORAN PRAKTIKUM
APLIKASI WEB DAN MOBILE

Semester Genap 2025/2026

MODUL 8 – Asynchronous JavaScript (Fetch API, JSON, REST API Sederhana)

Nama Mahasiswa : Shannaz Fairuz
NIM : 23051430020
Kelas : A1
Tanggal Praktikum : 17 April 2026
Dosen Pengampu : Dr. Eng. Ir. Aji Ery Burhandenny, S.T., M.AIT.

Dikumpulkan paling lambat 7 hari setelah pelaksanaan praktikum

A. IDENTITAS & INFORMASI MODUL

Mata Kuliah	Aplikasi Web dan Mobile
Kode Modul	Modul 8 (Pertemuan 8)
Topik Utama	Asynchronous JavaScript (Fetch API, JSON, REST API Sederhana)
Sub-Topik	Konsep Asynchronous, Fetch API, Promise, Async/Await, dan integrasi REST API eksternal.
Durasi Praktikum	340 Menit
Tanggal Praktikum	17 April 2026
Nama Mahasiswa	Shannaz Fairuz
NIM	23051430020
Kelas	A1

B. TUJUAN PEMBELAJARAN

1	Memahami perbedaan kode sinkronus dan asinkronus.
2	Mampu menggunakan <code>fetch()</code> untuk mengambil data dari server eksternal (API).
3	Memahami format data JSON (JavaScript Object Notation).
4	Menerapkan penanganan error saat request API gagal.

C. ALAT DAN BAHAN

No.	Nama Alat / Bahan / Aplikasi	Versi / Spesifikasi	Keterangan
1	Visual Studio Code (VS Code)	Versi 1.8x ke atas	Code Editor
2	Google Chrome / Browser Modern	Versi terbaru	Testing & Debugging
3	Node.js & npm	Node 18+ / npm 9+	Runtime JS (Modul tertentu)
4	JSONPlaceholder API	https://jsonplaceholder.typicode.com/	Sumber data API
5	Bootstrap	Versi 5.3 (CDN)	Framework CSS untuk tampilan

D. DASAR TEORI

D.1 Konsep Utama

Dipelajari konsep konsep JavaScript asynchronous, yaitu cara menjalankan kode tanpa harus menunggu proses sebelumnya selesai. Hal ini penting terutama ketika mengambil data dari server menggunakan API. Jika menggunakan cara sinkronus, proses akan berhenti sampai data selesai diambil, sedangkan asynchronous memungkinkan program tetap berjalan. Salah satu metode yang digunakan adalah Fetch API, yaitu fungsi bawaan JavaScript untuk mengambil data dari server. Data yang diterima biasanya dalam format JSON (JavaScript Object Notation) yang mudah dibaca dan diolah.

D.2 Keterkaitan dengan Teknik Industri

Pengolahan data dan sistem informasi perusahaan. Banyak sistem industri seperti ERP, dashboard produksi, dan monitoring menggunakan API untuk mengambil data secara real-time. Contohnya, data produksi, laporan kerusakan mesin, atau data karyawan dapat diambil dari server menggunakan API dan ditampilkan secara langsung ke dashboard. Hal ini membantu pengambilan keputusan menjadi lebih cepat dan akurat. Selain itu, penggunaan asynchronous memungkinkan sistem tetap responsif walaupun mengambil data dalam jumlah besar, sehingga meningkatkan efisiensi kerja dalam industri.

E. LANGKAH KERJA DAN HASIL PRAKTIKUM

E.1 Langkah 1 – Menenal JSONPlaceholder & Membuat HTML untuk Menampilkan Data

Deskripsi singkat apa yang dilakukan pada langkah ini:

Langkah pertama buat HTML untuk menampilkan data karyawan, lalu gunakan fetch dengan metode then() untuk mengambil data dari API JSONPlaceholder. Lalu digunakann async/await untuk mengambil data berdasarkan id

```

/* ===== KODE / OUTPUT LANGKAH 1 ===== */
!DOCTYPE html>
<html lang="id">
<head>
<meta charset="UTF-8">
<title>Integrasi API Data Karyawan</title>
<link
href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.0/dist/css/bootstrap.min.css"
rel="stylesheet">
</head>
<body class="container mt-5">
<h2 class="mb-4">Data Karyawan (Dari Server)</h2>
<div class="mb-3">
<button id="btnLoad" class="btn btn-primary">Muat Data dari API</button>
<div id="loading" class="spinner-border text-primary d-none" role="status"></div>
</div>
<div class="row" id="containerKaryawan">
<!-- Card karyawan akan muncul di sini -->
</div>
<script src="api_script.js"></script>
</body>
</html>

```

E.2 Langkah 2 – Fetch Data dengan .then() (Promise)

```

/* ===== KODE / OUTPUT LANGKAH 2 ===== */
[const btnLoad = document.getElementById('btnLoad'); const container =
document.getElementById('containerKaryawan'); const loading =
document.getElementById('loading'); // Endpoint API (Simulasi Database) const
API URL = 'https://jsonplaceholder.typicode.com/users';
btnLoad.addEventListener('click', function() { // Tampilkan loading
loading.classList.remove('d-none'); container.innerHTML = ''; // Bersihkan konten
lama // Fetch Data fetch(API_URL).then(function(response) { // Cek jika response
sukses (kode 200-299) if (!response.ok) { } throw new Error('Gagal mengambil
data'); // Parsing data JSON return response.json(); })
.then(function(dataKaryawan) { // Data berhasil didapat console.log(dataKaryawan);
// Cek di console browser renderData(dataKaryawan); }) .catch(function(error) { //
Jika ada error (misal putus internet) container.innerHTML = `
Error: ${error.message}
`; }) .finally(function() { // Sembunyikan loading (baik sukses maupun gagal)
loading.classList.add('d-none'); }); }); function renderData(data) {
data.forEach(function(karyawan) { // Buat elemen card Bootstrap const col =
document.createElement('div'); col.className = 'col-md-4 mb-3'; col.innerHTML = `
${karyawan.name}
Email: ${karyawan.email}
Perusahaan: ${karyawan.company.name}
Kota: ${karyawan.address.city}
Detail Profil
`; container.appendChild(col); });

```

E.3 Langkah 3 – Fetch dengan Async/Await (Modern Style)

```
/* ===== KODE / OUTPUT LANGKAH 3 ===== */
// Fungsi Async untuk mencari data spesifik
async function cariKaryawan(id) {
  try {
    console.log(`Mencari data ID: ${id}...`);

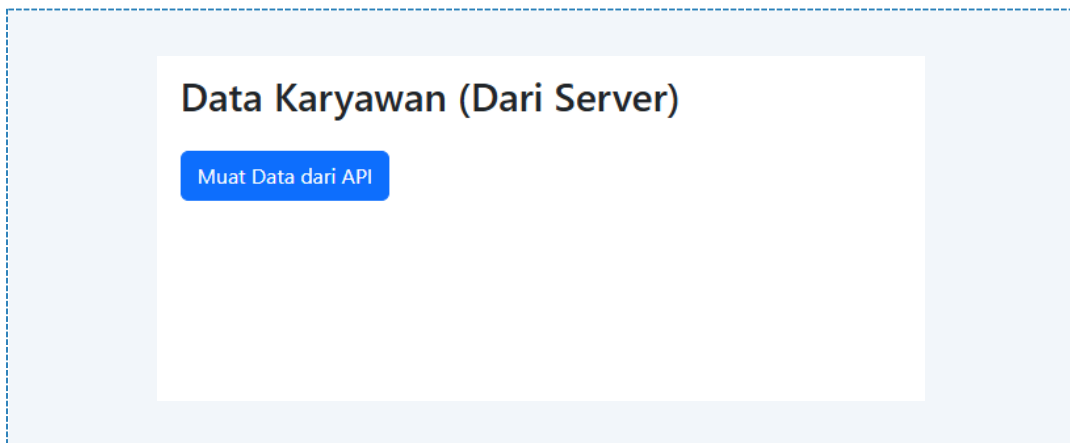
    const response = await
    fetch(`https://jsonplaceholder.typicode.com/users/${id}`);

    if (!response.ok) {
      throw new Error('Data tidak ditemukan');
    }

    const data = await response.json();
    console.log("Ditemukan:", data);
    alert(`Ditemukan: ${data.name} - bekerja di ${data.company.name}`);

  } catch (error) {
    console.error(error);
    alert(error.message);
  }
}
```

Screenshot Hasil:



Gambar Praktikum – [Praktikum]

F. LATIHAN DAN TUGAS MANDIRI

F.1 Latihan 1 – [Posting Data]

Pertanyaan / Instruksi Latihan:

Buat tombol "Tambah Karyawan Baru". Gunakan method `fetch` dengan opsi `{ method: 'POST', body: ... }` untuk mengirim data dummy ke API. Cek response di console.

Jawaban / Kode Solusi:

```
[HTML]
<!DOCTYPE html>
<html lang="id">
<head>
<meta charset="UTF-8">
<title>Integrasi API Data Karyawan</title>
<link
href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.0/dist/css/bootstrap.min.css"
rel="stylesheet">
</head>
<body class="container mt-5">
<h2 class="mb-4">Data Karyawan (Dari Server)</h2>
<div class="mb-3">
<button id="btnLoad" class="btn btn-primary me-2">Muat Data dari API</button>
<button id="btnTambah" class="btn btn-success">Tambah Karyawan Baru</button>
<div id="loading" class="spinner-border text-primary d-none" role="status"></div>
</div>
<div class="row" id="containerKaryawan">
<!-- Card karyawan akan muncul di sini -->
</div>
<script src="Latihan1.js"></script>
</body>
</html>

[SCRIPT JS]
const btnLoad = document.getElementById('btnLoad');
const btnTambah = document.getElementById('btnTambah'); // ← BARU
const container = document.getElementById('containerKaryawan');
const loading = document.getElementById('loading');

// Endpoint API (Simulasi Database)
const API_URL = 'https://jsonplaceholder.typicode.com/users';

btnLoad.addEventListener('click', function() {
  // Tampilkan loading
  loading.classList.remove('d-none');
  container.innerHTML = ''; // Bersihkan konten lama

  // Fetch Data
  fetch(API_URL)
    .then(function(response) {
      if (!response.ok) {
        throw new Error('Gagal mengambil data');
      }
      return response.json();
    })
    .then(function(dataKaryawan) {
      console.log(dataKaryawan);
      renderData(dataKaryawan);
    })
    .catch(function(error) {
      container.innerHTML = `<div class="alert alert-danger">Error:
${error.message}</div>`;
    })
    .finally(function() {
      loading.classList.add('d-none');
    });
});
```

```
});  
});  
  
// ← FUNGSI BARU: Tambah Karyawan (POST)  
btnTambah.addEventListener('click', function() {  
  // Data dummy karyawan baru  
  const karyawanBaru = {  
    name: 'Budi Santoso',  
    email: 'budi.santoso@perusahaan.com',  
    phone: '081234567890',  
    website: 'budisantoso.com',  
    company: {  
      name: 'PT Teknologi Maju',  
      catchPhrase: 'Solusi Inovatif',  
      bs: 'Enterprise Solutions'  
    },  
    address: {  
      street: 'Jl. Sudirman No. 123',  
      suite: 'Lantai 5',  
      city: 'Jakarta',  
      zipcode: '12190'  
    }  
  };  
  
  // Tampilkan loading  
  loading.classList.remove('d-none');  
  
  // POST Request dengan fetch  
  fetch(API_URL, {  
    method: 'POST',  
    headers: {  
      'Content-Type': 'application/json',  
    },  
    body: JSON.stringify(karyawanBaru) // ← Convert object ke JSON string  
  })  
  .then(function(response) {  
    if (!response.ok) {  
      throw new Error('Gagal menambah karyawan');  
    }  
    return response.json(); // JSONPlaceholder akan mengembalikan data yang  
    kita kirim + id=201  
  })  
  .then(function(dataResponse) {  
    console.log('✅ BERHASIL MENAMBAH KARYAWAN!');  
    console.log('Data yang dikirim:', karyawanBaru);  
    console.log('Response dari server:', dataResponse);  
  
    // Tampilkan notifikasi sukses  
    container.innerHTML = `  
      <div class="alert alert-success alert-dismissible fade show"  
role="alert">  
        <strong>Berhasil!</strong> Karyawan baru ditambahkan dengan ID:  
        ${dataResponse.id}  
        <button type="button" class="btn-close" data-bs-  
dismiss="alert"></button>  
      </div>  
    `;  
  
    // Auto reload data setelah 2 detik  
    setTimeout(() => {  
      btnLoad.click(); // Reload data otomatis  
    }, 2000);  
  })  
  .catch(function(error) {
```

```

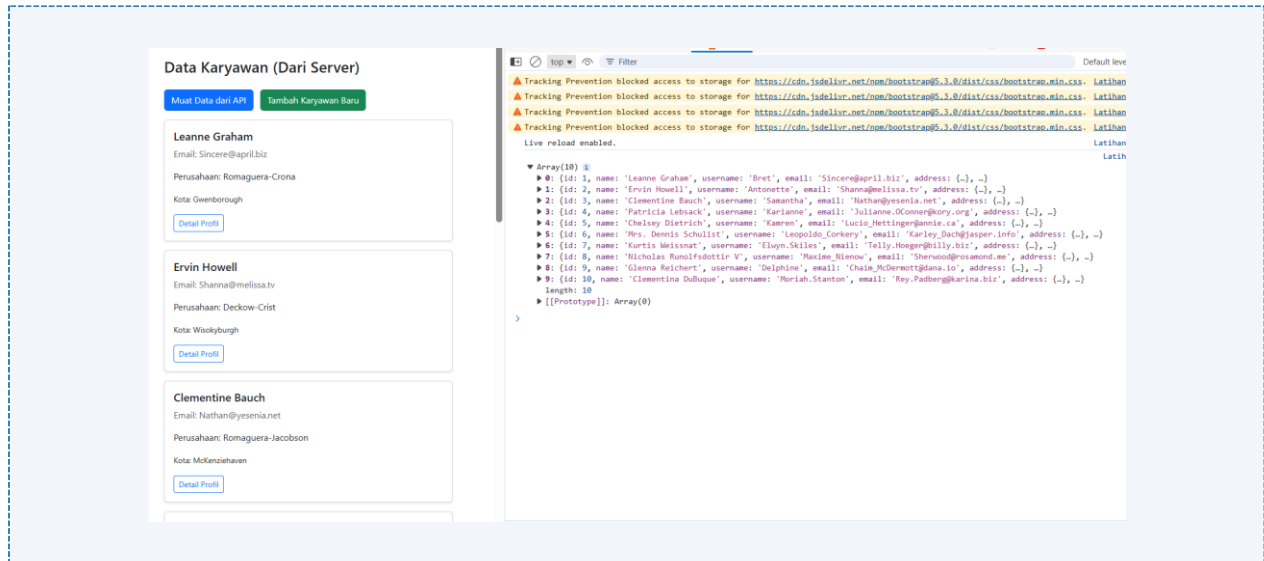
        console.error('✖ ERROR POST:', error);
        container.innerHTML = `
            <div class="alert alert-danger alert-dismissible fade show"
role="alert">
                <strong>Error!</strong> ${error.message}
                <button type="button" class="btn-close" data-bs-
dismiss="alert"></button>
            </div>
        `;
    })
    .finally(function() {
        loading.classList.add('d-none');
    });
});

function renderData(data) {
    data.forEach(function(karyawan) {
        const col = document.createElement('div');
        col.className = 'col-md-4 mb-3';

        col.innerHTML = `
            <div class="card h-100 shadow-sm">
                <div class="card-body">
                    <h5 class="card-title">${karyawan.name}</h5>
                    <p class="card-text text-muted">Email: ${karyawan.email}</p>
                    <p class="card-text">Perusahaan: ${karyawan.company.name}</p>
                    <p class="card-text"><small>Kota:
${karyawan.address.city}</small></p>
                    <a href="#" class="btn btn-sm btn-outline-primary"
onclick="cariKaryawan(${karyawan.id})">Detail Profil</a>
                </div>
            </div>
        `;
        container.appendChild(col);
    });
}

// Fungsi cari karyawan (sudah ada, ditambahkan onclick)
async function cariKaryawan(id) {
    try {
        console.log(`Mencari data ID: ${id}...`);
        const response = await
fetch(`https://jsonplaceholder.typicode.com/users/${id}`);
        if (!response.ok) {
            throw new Error('Data tidak ditemukan');
        }
        const data = await response.json();
        console.log("Ditemukan:", data);
        alert(`Ditemukan: ${data.name} - bekerja di ${data.company.name}`);
    } catch (error) {
        console.error(error);
        alert(error.message);
    }
}

```

Gambar F.1 – [Posting Data]

F.2 Latihan 2 – [Filter Array]

Pertanyaan / Instruksi Latihan:

Sebelum data dirender ke layar dengan `forEach`, lakukan filter terlebih dahulu. Tampilkan HANYA karyawan yang tinggal di kota yang mengandung huruf "s" (misal: "Southside", "London"). Petunjuk: Gunakan `data.filter(...)` sebelum `data.forEach(...)`.

Jawaban / Kode Solusi:

```
[HTML]
<!DOCTYPE html>
<html lang="id">
<head>
<meta charset="UTF-8">
<title>Integrasi API Data Karyawan</title>
<title> Shannaz Fairuz</title>
<link
href="https://cdn.jsdelivrivr.net/npm/bootstrap@5.3.0/dist/css/bootstrap.min.css"
rel="stylesheet">
</head>
<body class="container mt-5">
<h2 class="mb-4">Data Karyawan (Dari Server)</h2>
<div class="mb-3">
<button id="btnLoad" class="btn btn-primary me-2">Muat Data dari API</button>
<button id="btnTambah" class="btn btn-success me-2">Tambah Karyawan Baru</button>
<!-- INPUT FILTER BARU -->
<input type="text" id="filterKota" class="form-control w-auto d-inline-block me-2"
placeholder="Filter kota (ketik 's' untuk cari kota berawalan 's') "
maxlength="1">
<button id="btnFilter" class="btn btn-info">Filter</button>
<div id="loading" class="spinner-border text-primary d-none" role="status"></div>
</div>
<div class="row" id="containerKaryawan">
<!-- Card karyawan akan muncul di sini -->
</div>
<script src="Latihan2.js"></script>
</body>
</html>

[SCRIPT JS]
```

```
const btnLoad = document.getElementById('btnLoad');
const btnTambah = document.getElementById('btnTambah');
const btnFilter = document.getElementById('btnFilter'); // ← BARU
const inputFilter = document.getElementById('filterKota'); // ← BARU
const container = document.getElementById('containerKaryawan');
const loading = document.getElementById('loading');

const API_URL = 'https://jsonplaceholder.typicode.com/users';

// Event listener yang sudah ada (btnLoad & btnTambah) - TIDAK BERUBAH
btnLoad.addEventListener('click', function() {
  loading.classList.remove('d-none');
  container.innerHTML = '';

  fetch(API_URL)
    .then(function(response) {
      if (!response.ok) {
        throw new Error('Gagal mengambil data');
      }
      return response.json();
    })
    .then(function(dataKaryawan) {
      console.log('📄 Data mentah (10 karyawan):', dataKaryawan.length);
      renderData(dataKaryawan);
    })
    .catch(function(error) {
      container.innerHTML = `<div class="alert alert-danger">Error:
${error.message}</div>`;
    })
    .finally(function() {
      loading.classList.add('d-none');
    });
});

btnTambah.addEventListener('click', function() {
  const karyawanBaru = {
    name: 'Budi Santoso',
    email: 'budi.santoso@perusahaan.com',
    phone: '081234567890',
    website: 'budisantoso.com',
    company: { name: 'PT Teknologi Maju', catchPhrase: 'Solusi Inovatif', bs:
'Enterprise Solutions' },
    address: { street: 'Jl. Sudirman No. 123', suite: 'Lantai 5', city:
'Jakarta', zipcode: '12190' }
  };

  loading.classList.remove('d-none');

  fetch(API_URL, {
    method: 'POST',
    headers: { 'Content-Type': 'application/json' },
    body: JSON.stringify(karyawanBaru)
  })
    .then(response => {
      if (!response.ok) throw new Error('Gagal menambah karyawan');
      return response.json();
    })
    .then(dataResponse => {
      console.log(' BERHASIL MENAMBAH KARYAWAN!', dataResponse);
      container.innerHTML = `
        <div class="alert alert-success alert-dismissible fade show">
          <strong>Berhasil!</strong> Karyawan baru ditambahkan (ID:
${dataResponse.id})
      `;
    });
});
```

```

        <button type="button" class="btn-close" data-bs-
dismiss="alert"></button>
    </div>
    `;
    setTimeout(() => btnLoad.click(), 2000);
  })
  .catch(error => {
    console.error(' ERROR POST:', error);
    container.innerHTML = `<div class="alert alert-danger">Error:
${error.message}</div>`;
  })
  .finally(() => loading.classList.add('d-none'));
});

// ← FUNGSI BARU: Filter berdasarkan input
btnFilter.addEventListener('click', function() {
  const filterText = inputFilter.value.toLowerCase().trim();
  if (filterText) {
    console.log(` Mencari kota mengandung: "${filterText}"`);
    btnLoad.click(); // Reload dan filter otomatis
  }
});

// Enter key di input filter
inputFilter.addEventListener('keypress', function(e) {
  if (e.key === 'Enter') {
    btnFilter.click();
  }
});

// ← FUNGSI renderData DIPERBAIKI DENGAN FILTER!
function renderData(data) {
  const filterText = inputFilter.value.toLowerCase().trim();

  // FILTER: Hanya tampilkan kota yang MENGANDUNG huruf tertentu
  let dataFiltered;
  if (filterText) {
    dataFiltered = data.filter(function(karyawan) {
      // Cek apakah kota mengandung huruf yang dicari (case insensitive)
      return karyawan.address.city.toLowerCase().includes(filterText);
    });
    console.log(` Filter "${filterText}": ${dataFiltered.length}/${data.length}
karyawan ditemukan`);
  } else {
    dataFiltered = data; // Tidak ada filter
    console.log(` Semua data: ${dataFiltered.length} karyawan`);
  }

  // Bersihkan container
  container.innerHTML = '';

  if (dataFiltered.length === 0) {
    container.innerHTML = `
      <div class="col-12">
        <div class="alert alert-warning text-center">
          <h5>Tidak ada karyawan ditemukan</h5>
          <p>Kota yang mengandung "${filterText}" tidak ditemukan</p>
          <button class="btn btn-outline-primary"
onclick="inputFilter.value='';btnLoad.click()">
            Hapus Filter & Muat Ulang
          </button>
        </div>
      </div>
    `;
  }
};

```

```

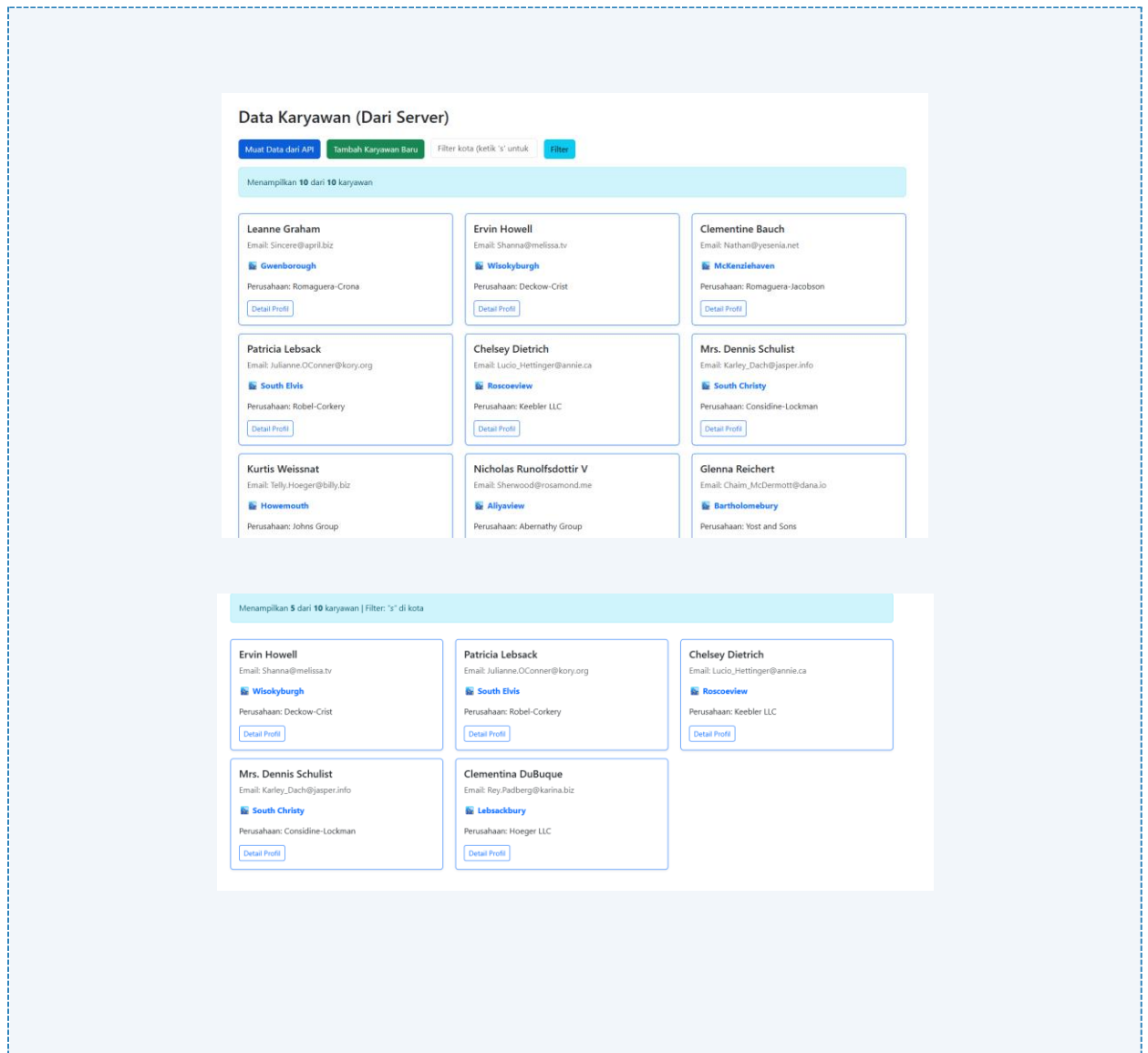
        return;
    }

    // Render filtered data
    dataFiltered.forEach(function(karyawan) {
        const col = document.createElement('div');
        col.className = 'col-md-4 mb-3';
        col.innerHTML = `
            <div class="card h-100 shadow-sm border-primary">
                <div class="card-body">
                    <h5 class="card-title">${karyawan.name}</h5>
                    <p class="card-text text-muted">Email: ${karyawan.email}</p>
                    <p class="card-text fw-bold text-primary">🏠
                    ${karyawan.address.city}</p>
                    <p class="card-text">Perusahaan: ${karyawan.company.name}</p>
                    <a href="#" class="btn btn-sm btn-outline-primary"
                    onclick="cariKaryawan(${karyawan.id})">Detail Profil</a>
                </div>
            </div>
        `;
        container.appendChild(col);
    });

    // Tampilkan info filter di atas
    const infoFilter = document.createElement('div');
    infoFilter.className = 'col-12 mb-3';
    infoFilter.innerHTML = `
        <div class="alert alert-info">
            Menampilkan <strong>${dataFiltered.length}</strong> dari
            <strong>${data.length}</strong> karyawan
            ${filterText ? `| Filter: <em>"${filterText}"</em> di kota` : ''}
        </div>
    `;
    container.insertBefore(infoFilter, container.firstChild);
}

// Fungsi cariKaryawan - TIDAK BERUBAH
async function cariKaryawan(id) {
    try {
        const response = await
        fetch(`https://jsonplaceholder.typicode.com/users/${id}`);
        if (!response.ok) throw new Error('Data tidak ditemukan');
        const data = await response.json();
        alert(`${data.name}\n ${data.email}\n ${data.company.name}\n
        ${data.address.city}`);
    } catch (error) {
        alert('Error: ' + error.message);
    }
}

```



Gambar F.2 – [Filter Array]

F.3 Tugas Proyek Mini – [Integrasi Data Produksi]

Deskripsi proyek mini:

- Anggap API `https://jsonplaceholder.typicode.com/posts` adalah database "Laporan Insiden".
- Tampilkan daftar 10 laporan insiden terbaru (Title = Judul Insiden, Body = Deskripsi).
 - Di setiap card, tambahkan tombol "Tindak Lanjut". Saat diklik, `alert()` menampilkan "Tiket ID sedang diproses oleh Tim Maintenance".

Penjelasan pendekatan/solusi yang digunakan:

API digunakan sebagai sumber data laporan insiden produksi. Langkah pertama adalah mengambil data dari endpoint menggunakan fetch, kemudian membatasi hanya 10 data pertama menggunakan .slice().

Selanjutnya, data tersebut dimodifikasi agar lebih relevan dengan konteks industri, misalnya mengganti isi teks dengan laporan insiden nyata. Setelah itu, data ditampilkan dalam bentuk card menggunakan fungsi render.

Setiap card ditambahkan tombol “Tindak Lanjut”, yang ketika diklik akan menampilkan notifikasi bahwa tiket sedang diproses. Selain itu, ditambahkan fitur pencarian dan filter untuk memudahkan pengguna dalam menemukan data tertentu.

Kode Utama:

```
[HTML]
!doctype html>
<html lang="id">
  <head>
    <meta charset="UTF-8" />
    <meta name="viewport" content="width=device-width, initial-scale=1.0" />
    <title>Sistem Pelaporan Insiden ERP</title>
    <link
      href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.0/dist/css/bootstrap.min.css"
      rel="stylesheet"
    />
    <link rel="stylesheet" href="Miniproyek.css" />
  </head>
  <body class="container mt-5">
    <div
      class="d-flex justify-content-between align-items-center mb-4 border-bottom
pb-3"
    >
      <div>
        <h2 class="text-dark fw-bold mb-0"> Dashboard Insiden Produksi</h2>
        <span class="badge bg-secondary mt-2">
          Penanggung Jawab: Shannaz Fairuz</span>
      </div>
      <div>
        <button id="btnLoadInsiden" class="btn btn-outline-secondary me-2">
          Muat Data Server
        </button>
        <button id="btnTambahInsiden" class="btn btn-corporate">
          Buat Tiket Baru
        </button>
      </div>
    </div>

    <div class="row mb-4 bg-white p-3 shadow-sm rounded-3 mx-0 border">
      <div class="col-md-8">
        <label class="form-label fw-bold text-secondary small">
          Pencarian Kata Kunci</label>
        <input
          type="text"
          id="inputSearch"
          class="form-control"
          placeholder="Cari berdasarkan judul atau deskripsi tiket..."
        />
      </div>
      <div class="col-md-4">
```

```

        <label class="form-label fw-bold text-secondary small"
          >Filter Lokasi</label
        >
        <select id="selectFilter" class="form-select">
          <option value="all">Semua Line Produksi</option>
          <option value="lineA">Line Produksi A (ID Ganjil)</option>
          <option value="lineB">Line Produksi B (ID Genap)</option>
        </select>
      </div>
    </div>

    <div id="loading" class="text-center d-none my-5">
      <div class="spinner-border text-primary" role="status"></div>
      <p class="text-muted mt-2 fw-semibold">Memproses data...</p>
    </div>

    <div class="row" id="containerInsiden"></div>

    <script src="Miniproyek.js"></script>
  </body>
</html>

[SCRIPT JS]
// Mengambil elemen HTML dan menyimpannya ke dalam variabel
const btnLoad = document.getElementById("btnLoadInsiden");
const btnTambah = document.getElementById("btnTambahInsiden");
const container = document.getElementById("containerInsiden");
const loading = document.getElementById("loading");

const inputSearch = document.getElementById("inputSearch");
const selectFilter = document.getElementById("selectFilter");

// Endpoint API yang diberikan oleh dosen
const API_URL = "https://jsonplaceholder.typicode.com/posts";

// Variabel Global untuk menyimpan data
let dataInsidenGlobal = [];

// =====
// DATA DUMMY INDONESIA (PENGGANTI LOREM IPSUM)
// =====
const laporanReal = [
  { title: "Pompa Air Pendingin Tidak Berfungsi", body: "Pompa utama di sistem pendingin unit 5 berhenti bekerja, menyebabkan suhu mesin naik dan produksi harus dihentikan sementara." },
  { title: "Lampu Penerangan Gudang Mati", body: "Beberapa lampu di area gudang bahan baku padam, mengganggu proses pengecekan stok dan distribusi barang." },
  { title: "Kompresor Udara Bocor", body: "Terdeteksi kebocoran pada pipa kompresor sektor C, tekanan udara tidak stabil sehingga mesin pneumatic terganggu." },
  { title: "Sensor Suhu Ruang Oven Error", body: "Indikator suhu pada panel kontrol oven nomor 3 tidak menunjukkan angka yang stabil, fluktuatif di luar batas normal." },
  { title: "Mesin Cetak Label Stuck", body: "Roll label macet di mesin cetak nomor 2, menghambat jalur produksi dan menyebabkan penundaan pengemasan." },
  { title: "Kualitas Bahan Baku Menurun", body: "Bahan baku dari supplier kloter pagi ini (batch #882) memiliki tingkat kelembapan di atas standar kualitas." },
  { title: "Kerusakan Forklift Gudang", body: "Garpu pengangkat pada Forklift unit 02 hidroliknya bocor sehingga tidak bisa mengangkat beban maksimal." },
  { title: "AC Ruang Kontrol Tidak Dingin", body: "AC di ruang kontrol utama tidak berfungsi dengan baik, suhu naik di atas batas nyaman untuk operator." },
  { title: "Kebakaran Kecil di Ruang Genset", body: "Terjadi korsleting ringan yang menimbulkan percikan api. Api sudah dipadamkan pakai APAR, tapi butuh perbaikan kabel." },
];

```

```

    { title: "Pintu Otomatis Loading Dock Rusak", body: "Sensor gerak pintu gerbang
pemuatan barang tidak merespon, pintu harus dibuka secara manual." },
    { title: "Kipas Ventilasi Meledak", body: "Kipas ventilasi di sektor finishing
mengalami kerusakan mekanik, menimbulkan percikan dan kebisingan tinggi." }
];

// =====
// 1. FUNGSI READ: Mengambil Data dari Server API
// =====
btnLoad.addEventListener("click", function () {
    loading.classList.remove("d-none");
    container.innerHTML = "";

    fetch(API_URL)
        .then((response) => {
            if (!response.ok) throw new Error("Gagal terhubung ke server ERP");
            return response.json();
        })
        .then((dataPosts) => {
            // Ambil 10 data dari API
            let sepuluhLaporan = dataPosts.slice(0, 10);

            // KODE SULAP: Ganti isi Lorem Ipsum dengan data bahasa Indonesia kita
            sepuluhLaporan = sepuluhLaporan.map((insiden, index) => {
                return {
                    id: insiden.id,
                    title: laporanReal[index].title,
                    body: laporanReal[index].body
                };
            });

            // Simpan ke variabel global
            dataInsidenGlobal = sepuluhLaporan;

            // Tampilkan ke layar
            renderListKeLayar(dataInsidenGlobal);
        })
        .catch((error) => {
            container.innerHTML = `<div class="alert alert-danger shadow-
sm"><strong>Sistem Error:</strong> ${error.message}</div>`;
        })
        .finally(() => {
            loading.classList.add("d-none");
        });
});

// =====
// 2. FUNGSI RENDER: Menggambar Kartu ke Layar HTML
// =====
function renderListKeLayar(arrayData) {
    container.innerHTML = "";

    if (arrayData.length === 0) {
        container.innerHTML = `<div class="col-12 text-center text-muted mt-4"><em>Tidak
ada tiket insiden yang sesuai dengan pencarian.</em></div>`;
        return;
    }

    arrayData.forEach((insiden) => {
        const lokasi = insiden.id % 2 === 0 ? "Line B" : "Line A";
        const badgeColor = insiden.id % 2 === 0 ? "bg-warning text-dark" : "bg-info
text-dark";

        const col = document.createElement("div");

```



```

col.className = "col-md-6 col-lg-4 mb-4";
col.id = `tiket-${insiden.id}`;

col.innerHTML = `
    <div class="card h-100 shadow-sm border-0 rounded-3">
        <div class="card-header card-header-corporate d-flex justify-
content-between align-items-center rounded-top-3" style="background-color: #0c2340;
color: white;">
            <span class="fw-bold">ID: #${insiden.id}</span>
            <span class="badge ${badgeColor}">${lokasi}</span>
        </div>
        <div class="card-body bg-white flex-column d-flex">
            <h6 class="card-title text-capitalize fw-bold text-
dark">${insiden.title}</h6>
            <p class="card-text text-secondary small mb-
3">${insiden.body}</p>
        </div>
        <div class="card-footer bg-light border-top-0 d-flex gap-2 rounded-
bottom-3">
            <button class="btn btn-sm btn-primary flex-fill fw-semibold"
onclick="tindakLanjut(${insiden.id})">Tindak Lanjut</button>
            <button class="btn btn-sm btn-outline-danger fw-semibold"
onclick="tutupTiket(${insiden.id})">Tutup</button>
        </div>
    </div>
`;
container.appendChild(col);
});
}

// =====
// 3. FUNGSI PENCARIAN & FILTER REALTIME
// =====
function jalankanSearchDanFilter() {
    const kataKunci = inputSearch.value.toLowerCase();
    const filterLokasi = selectFilter.value;

    const hasilFilter = dataInsidenGlobal.filter((insiden) => {
        const cocokTeks =
            insiden.title.toLowerCase().includes(kataKunci) ||
            insiden.body.toLowerCase().includes(kataKunci);

        let cocokLokasi = false;
        if (filterLokasi === "all") {
            cocokLokasi = true;
        } else if (filterLokasi === "lineA" && insiden.id % 2 !== 0) {
            cocokLokasi = true;
        } else if (filterLokasi === "lineB" && insiden.id % 2 === 0) {
            cocokLokasi = true;
        }

        return cocokTeks && cocokLokasi;
    });

    renderListKeLayar(hasilFilter);
}

inputSearch.addEventListener("input", jalankanSearchDanFilter);
selectFilter.addEventListener("change", jalankanSearchDanFilter);

// =====
// 4. FUNGSI ACTION, CREATE (TAMBAH), & DELETE (HAPUS)
// =====

```

```
function tindakLanjut(id) {
  alert(`[SISTEM ERP] Tiket ID #${id} sedang diproses oleh Tim Maintenance.`);
}

btnTambah.addEventListener("click", function () {
  const tiketBaru = {
    title: "Tumpahan Bahan Kimia (Data Baru)",
    body: "Drum penyimpanan bahan kimia H2O2 terguling. Area sudah diisolasi, butuh tim kebersihan khusus segera.",
    userId: 1,
  };
  loading.classList.remove("d-none");

  fetch(API_URL, {
    method: "POST",
    headers: { "Content-Type": "application/json" },
    body: JSON.stringify(tiketBaru),
  })
    .then((res) => res.json())
    .then((data) => {
      alert(`[SUKSES] Tiket baru dibuat dengan ID #${data.id}`);
      dataInsidenGlobal.unshift(data);
      renderListKeLayar(dataInsidenGlobal);
    })
    .catch((error) => alert("Gagal membuat tiket: " + error.message))
    .finally(() => loading.classList.add("d-none"));
});

function tutupTiket(id) {
  if (confirm(`Apakah Anda yakin ingin menutup tiket #${id}?`)) {
    dataInsidenGlobal = dataInsidenGlobal.filter((insiden) => insiden.id !== id);
    renderListKeLayar(dataInsidenGlobal);
  }
}

[STYLE CSS]
.hover-lift {
  transition: all 0.3s ease;
}

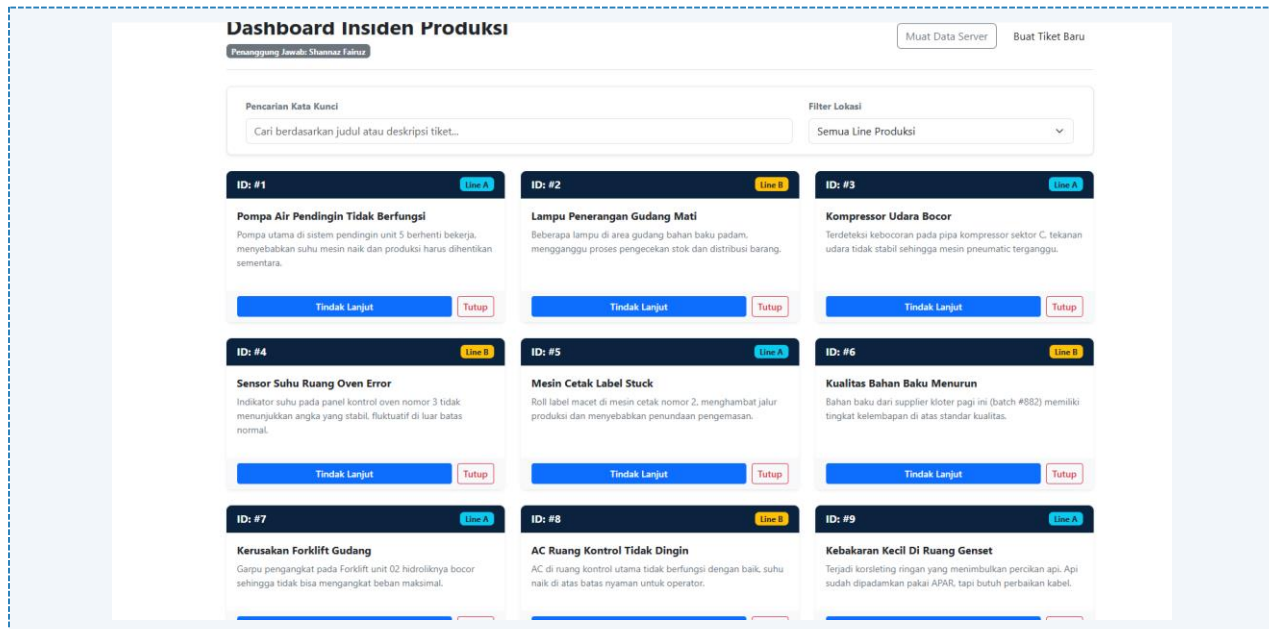
.hover-lift:hover {
  transform: translateY(-8px);
  box-shadow: 0 20px 40px rgba(0,0,0,0.15) !important;
}

.bg-gradient-primary {
  background: linear-gradient(135deg, #667eea 0%, #764ba2 100%) !important;
}

.card-header {
  position: relative;
  overflow: hidden;
}

.card-header::before {
  content: '';
  position: absolute;
  top: 0;
  left: -100%;
  width: 100%;
  height: 100%;
  background: linear-gradient(90deg, transparent, rgba(255,255,255,0.2),
transparent);
  transition: left 0.5s;
```

```
}  
  
.card:hover .card-header::before {  
  left: 100%;  
}  
}
```



Gambar F.3 – [Integrasi Data]

G. PEMBAHASAN

G.1 Analisis Kesesuaian Hasil dengan Teori

Pada langkah penggunaan `fetch()` dengan metode `.then()`, data dari API `https://jsonplaceholder.typicode.com/users` berhasil diambil dan ditampilkan dalam bentuk card karyawan di halaman web. Hal ini menunjukkan bahwa konsep Promise berjalan dengan baik, di mana data diproses setelah response diterima. Selain itu, penggunaan `async/await` pada fungsi pencarian karyawan juga sesuai dengan teori. Fungsi tersebut mampu mengambil data berdasarkan ID tertentu

G.2 Kendala yang Ditemukan dan Solusinya

Kendala / Error yang Ditemukan	Solusi yang Diterapkan
Data tidak muncul saat pertama dijalankan	Memastikan endpoint API benar dan koneksi internet stabil
Error saat parsing JSON	Mengecek penggunaan <code>response.json()</code> sudah benar
Tampilan tidak rapi	Menggunakan Bootstrap agar tampilan lebih terstruktur

G.3 Kaitan dengan Penerapan di Industri

data mesin produksi dapat diambil dari server dan ditampilkan secara real-time. Jika terjadi kerusakan, sistem dapat langsung memberikan notifikasi kepada operator. Selain itu, fitur filter dan pencarian sangat berguna untuk menganalisis data dalam jumlah besar, sehingga membantu efisiensi dan pengambilan keputusan di perusahaan.

H. KESIMPULAN

1	JavaScript asynchronous memungkinkan pengambilan data tanpa menghambat proses program.
2	Fetch API dapat digunakan untuk mengambil data dari server dalam format JSON.
3	Async/await lebih mudah digunakan dibandingkan .then() dalam penulisan kode.
4	Data dari API dapat ditampilkan ke halaman web secara dinamis dan interaktif.
5	Konsep ini sangat berguna dalam pengembangan sistem informasi di dunia industri.

I. DAFTAR PUSTAKA

No.	Referensi (Format APA 7th Edition)
[1]	Burhandenny, A. E. (2026). Manual Praktikum Aplikasi Web dan Mobile Semester Genap 2025/2026. Program Studi Teknik Industri, Universitas Negeri Yogyakarta.
[2]	Mozilla Developer Network. (2024). <i>Fetch API</i> . https://developer.mozilla.org
[3]	W3Schools. (2024). <i>JavaScript JSON</i> . https://www.w3schools.com
[4]	JSONPlaceholder. (2024). <i>Fake Online REST API</i> . https://jsonplaceholder.typicode.com

J. LEMBAR PENILAIAN DAN PENGESAHAN

RUBRIK PENILAIAN LAPORAN					
Aspek Penilaian		Bobot			
No.	Aspek Penilaian	Bobot (%)	Nilai (0-100)	Skor Akhir	Catatan Penilai
1	Kelengkapan Isi (Semua bagian A–I terisi lengkap)	20%			
2	Kualitas Dasar Teori (Ditulis dengan bahasa sendiri, relevan, minimal 3 paragraf)	15%			
3	Dokumentasi Langkah Kerja (Kode & Screenshot sesuai, jelas, dan terbaca)	25%			
4	Tugas/Latihan Mandiri (Semua latihan & proyek mini dikerjakan dan berfungsi)	20%			
5	Pembahasan & Analisis (Kritis, mendalam, dan relevan dengan industri)	15%			
6	Kesimpulan & Tata Bahasa (Spesifik, rapi, mengikuti format yang ditentukan)	5%			
TOTAL NILAI AKHIR		100%			

Mahasiswa	Diperiksa oleh Asisten / Dosen
_____ Shannaz Fairuz NIM: [23051430020]	_____ Dr. Eng. Ir. Aji Ery Burhandenny, S.T., M.AIT. Tanggal: _____